



Homework 5

Please complete the following problems. You must submit the following for each problem: a) the answer you are asked to return and b) Python code that can be run to verify your work.

1. Global Alignment with Scoring Matrix

In the “Edit Distance” problem, the edit distance counted the total number of edit operations implied by an alignment, where an edit operation was defined as the substitution, insertion, or deletion of a single symbol.

The related edit alignment score is an alignment scoring function that assigns a (negative) unit cost to every edit operation appearing in the alignment and a cost of zero to every matched symbol. Note that every edit operation in the alignment corresponds to a mismatched symbol (where a mismatched gap symbol corresponds to an insertion or deletion).

A very small alignment of DNA strings is shown in Figure 1. Note that this alignment implies two deletions, one insertion, and one substituted nucleotide. There are 4 total positions at which symbols do not match, so that the edit score of the alignment is -4.

A	A	C	G	T	-	A
A	-	-	G	T	C	T

Figure 1: Example alignment with an edit score of -4.

Another common scoring function awards matched symbols with 1 and penalizes substituted/inserted/deleted symbols equally by assigning each one a score of 0, so that the maximum score of an alignment becomes the length of a longest common subsequence of s and t . In general, the alignment score is simply a scoring function that assigns costs to edit operations encoded by the alignment.

One natural way of adding complexity to alignment scoring functions is by changing the alignment score based on which symbols are substituted; many methods have been proposed for doing this. Another way to do so is to vary the penalty assigned to the insertion or deletion of symbols.

In general, alignment scores can be either maximized or minimized depending on how scores are established. The general problem of optimizing a particular alignment score is called global alignment.

PROBLEM

To penalize symbol substitutions differently depending on which two symbols are involved in the substitution, we obtain a scoring matrix S in which $S_{i,j}$ represents the (negative) score assigned to a substitution of the i th symbol of our nucleotide/amino acid code with the j th symbol.



A gap penalty is the component deducted from alignment score due to the presence of a gap. A gap penalty may be a function of the length of the gap; for example, a linear gap penalty is a constant g such that each inserted or deleted symbol is charged g ; as a result, the cost of a gap of length L is equal to gL .

Given: A file containing two protein strings s and t in FASTA format (each of length at most 1000 aa).

Return: The maximum alignment score between s and t . Use:

- The BLOSUM62 scoring matrix.
- Linear gap penalty equal to 5 (i.e., a cost of -5 is assessed for each gap symbol).

Hint: There are many sequence alignment algorithms, both pairwise and multiple sequence alignments. These calculations are relatively slow, and you generally would not want to write such algorithms in Python.

Therefore, the simplest way to solve this problem in Python is to use the pairwise alignment algorithm **PairwiseAligner** in Biopython. The **Bio.Align** module contains the **PairwiseAligner** class for global and local alignments using the Needleman-Wunsch, Smith-Waterman, Gotoh (three-state), and Waterman-Smith-Beyer global and local pairwise alignment algorithms, with numerous options to change the alignment parameters. Please see the [Biopython Tutorial and Cookbook](#) for details on pairwise alignment in Python. For this problem, you only need to set the scoring matrix and gap score as described above; the default alignment mode is global.



2. Local Alignment with Scoring Matrix

Whereas global alignment (see the “Global Alignment with Scoring Matrix” problem) can be helpful for comparing genetic strings of similar length that resemble each other, often we will be presented with strings that are mostly dissimilar except for some unknown region of the strings, which may represent a shared gene. To find such genes, we need to modify global alignment to instead search for shared motifs in the form of locally similar regions (recall the “Finding a Shared Motif” problem).

Using global alignment often fails to find shared motifs hidden in larger strings, especially if the similar region is found on different ends of the string, because aligning the strings causes gap penalties to rack up.

If we are only interested in comparing the regions of similarity, then we would like to have some way of disregarding the parts of the strings that don't resemble each other. The way to do this is to produce alignment scores for all possible pairs of substrings.

PROBLEM

A local alignment of two strings s and t is an alignment of substrings r and u of s and t , respectively. Let $opt(r, u)$ denote the score of an optimal alignment of r and u with respect to some predetermined alignment score.

Given: A file containing two protein strings s and t in FASTA format (each having length at most 1000 aa).

Return: A maximum alignment score along with substrings r and u of s and t , respectively, which produce this maximum alignment score (multiple solutions may exist, in which case you may output any one). Use:

- The PAM250 scoring matrix.
- Linear gap penalty equal to 5 (i.e., a cost of -5 is assessed for each gap symbol).

Sample Dataset

```
>Rosalind_80
MEANLYPRTEINSTRING
>Rosalind_21
PLEASANTLYEINSTEIN
```

Sample Output

```
23
LYPRTEINSTRIN
LYEINSTEIN
```

Hint: You should again use the pairwise alignment algorithm `PairwiseAligner` in Biopython, which contains local alignment algorithms. Please see the [Biopython Tutorial and Cookbook](#) for details on pairwise alignment in Python. For this problem, you need to change the alignment mode and set the scoring matrix and gap score. The `aligner.align` method returns Alignment objects, each representing one alignment between two sequences. To get the aligned sequence strings individually, as asked for in the problem, use `alignment[0]` for the aligned target sequence and `alignment[1]` for the query. You can then use the Python `replace` method to remove the dashes from the sequences.



3. Multiple Alignment

The Hamming distance between two strings having the same length is the minimum number of symbol substitutions required to transform one string into the other. The Hamming distance only works if two sequences have the same length.

As noted in the “Edit Distance” problem, homologous strands of DNA have varying lengths because of the effect of mutations inserting and deleting intervals of genetic material. The edit distance is the minimum number of edit operations needed to transform one string into the other, where an edit operation is defined as the substitution, insertion, or deletion of a single symbol.

For a certain string, its augmented string is formed by adding gap symbols to it. For the strings $s = \text{ACGTA}$ and $t = \text{AGTCT}$, we may add gap symbols to s and t to produce the augmented strings $\bar{s} = \text{ACGT-A}$ and $\bar{t} = \text{A--GTCT}$.

Assuming two augmented strings have the same length, they may be compared by aligning them, which offers a scenario by which one string could have been transformed into another via the substitution, insertion, and deletion of symbols. We assume that gap symbols from the two augmented strings are not found in corresponding positions. Gap symbols in $\bar{s}$ are matched against symbols in $\bar{t}$ that were inserted into s ; gap symbols in $\bar{t}$ are matched against symbols in $\bar{s}$ that were deleted from s .

For example, let us reconsider the augmented strings $\bar{s}$ (top) and $\bar{t}$ (bottom) in Figure 1. The simplest way of inferring a collection of point mutations transforming s into t is to assume that the 'A' and 'C' from the second and third positions of $\bar{s}$ were deleted, a 'C' was inserted into the sixth position of $\bar{s}$, and the final symbol was changed from an 'A' to a 'T'.



Figure 1: Augmented strings $\bar{s}$ (top) and $\bar{t}$ (bottom). Substitutions are encoded in red, insertions/deletions in blue; and fixed symbols in green.

A multiple alignment of a collection of strings is a generalization of the notion of alignment to more than two strings. A multiple alignment is formed by adding gap symbols to all strings to produce a collection of augmented strings having the same length.

A multiple alignment score is obtained by taking the sum of an alignment score over all possible pairs of augmented strings. The only difference in scoring the alignment of two strings is that two gap symbols may be aligned for a given pair (requiring us to specify a score for matched gap symbols).

**PROBLEM**

Again, alignment calculations are relatively slow, and you generally would not want to write such algorithms in Python. (Although an algorithm to solve this problem would not be slow since there are only four strings of ≤ 10 bp.) But feel free to solve the problem this way if you desire.

There is no multiple alignment algorithm in Biopython akin to the **PairwiseAligner** algorithm. It is possible to run some external MSA algorithms in Biopython, but it is much simpler to download the programs and run them locally on your command line. Generally, command line programs are more fully featured than web-based programs, and it is valuable to feel comfortable running bioinformatics software from the command line.

The links to download the MSA programs MUSCLEv5 and Clustal Omega can be found above the input files on the ELMS assignment. The program files have been verified and are safe to download/install.

Given: A file containing a collection of four DNA strings of length, each being at most 10 bp and in FASTA format.

Return: A multiple alignment of the strings in FASTA format generated by MUSCLEv5 and Clustal Omega using their default parameters. (Two alignments should be submitted.)

Note: instead of code, please submit screenshot(s) of the MUSCLEv5 and Clustal Omega commands being run on your command line.

Sample Dataset

```
>Rosalind_7
ATATCCG
>Rosalind_35
TCCG
>Rosalind_23
ATGTACTG
>Rosalind_44
ATGTCTG
```

Sample Output

```
>Rosalind_7
ATAT-CCG
>Rosalind_35
-T---CCG
>Rosalind_23
ATGTACTG
>Rosalind_44
ATGT-CTG
```

Hint: Both algorithms output the alignment in FASTA format by default. Note: even with extremely simple input sequences, the two MSA algorithms output different alignments using their default parameters.



4. Computing GC Content

A quick method used by early computer software to determine the language of a given piece of text was to analyze the frequency with which each letter appeared in the text. This strategy was used because each language tends to exhibit its own letter frequencies, and if the text under consideration is long enough, software will correctly recognize the language quickly and with a very low error rate. See Figure 1 for English letter frequencies.

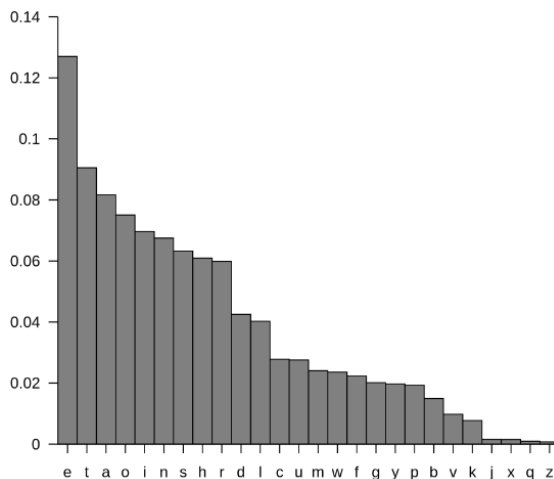


Figure 1: The table above was computed from many English words and shows for any letter the frequency with which it appears in those words. These frequencies can be used to reliably identify a piece of English text and differentiate it from that of another language. Taken from http://en.wikipedia.org/wiki/File:English_letter_,

You may ask: what in the world does this linguistic problem have to do with biology? Although two members of the same species will have different genomes, they still share the vast percentage of their DNA; notably, 99.9% of the 3.2 billion base pairs in a human genome are common to almost all humans (i.e., excluding people having major genetic defects). For this reason, biologists will speak of the human genome, meaning an average-case genome derived from a collection of individuals. Such an average case genome can be assembled for any species, a challenge that we will soon discuss.

The biological analog of identifying unknown text arises when researchers encounter a molecule of DNA from an unknown species. Because of the base pairing relations of the two DNA strands, cytosine and guanine will always appear in equal amounts in a double-stranded DNA molecule. Thus, to analyze the symbol frequencies of DNA for comparison against a database, we compute the molecule's GC-content, or the percentage of its bases that are either cytosine or guanine.

In practice, the GC-content of most eukaryotic genomes hovers around 50%. However, because genomes are so long, we may be able to distinguish species based on very small discrepancies in GC-content; furthermore, most prokaryotes have a GC-content significantly higher than 50%, so that GC-content can be used to quickly differentiate many prokaryotes and eukaryotes by using relatively small DNA samples.

PROBLEM

The GC-content of a DNA string is given by the percentage of symbols in the string that are 'C' or 'G'. For example, the GC-content of "AGCTATAG" is 37.5%. Note that the reverse complement of any DNA string has the same GC-content.



DNA strings must be labeled when they are consolidated into a database. A commonly used method of string labeling is the FASTA format. As a reminder, each string in a FASTA file is introduced by a line that begins with '>', followed by some labeling information. Subsequent lines contain the string itself; the first line to begin with '>' indicates the label of the next string.

Given: A file containing at most 10 DNA strings in FASTA format, with each string of length at most 1 kb.

Return: The ID of the string having the highest GC-content, followed by the GC-content of that string. There is a default error of 0.001 in all decimal answers unless otherwise stated; please see the note on absolute error below. For example, if an exact solution is 6.157892, then for x to be within an absolute error of 0.001, we must have that $|x - 6.157892| < 0.001$, or $6.156892 < x < 6.158892$.

Sample Dataset

```
>Rosalind_6404
CCTGCGGAAGATCGGCACTAGAAATAGCCAGAACCGTTTCTCTGAGGCTTCCGGCCTTCCC
TCCCATAATAATTCTGAGG
>Rosalind_5959
CCATCGGTAGCGCATCCTTAGTCCAATTAAGTCCCTATCCAGGCGCTCCGCCGAAGGTCT
ATATCCATTTGTCAGCAGACACGC
>Rosalind_0808
CCACCCTCGTGGTATGGCTAGGCATTAGGAACCGGAGAACGCTTCAGACCAGCCCGGAC
TGGGAACCTGCGGGCAGTAGGTGGAAT
```

Sample Output

```
Rosalind_0808
60.919540
```

Hint: You can perform the simple GC percentage calculation in your code, or you can use the `gc_fraction` function, which must be imported from the `Bio.SeqUtils` module of Biopython.